Lab 3: NYC Taxi Data Profiling & Cleaning

Name: Ahmed Tarek Mohamed | **Student ID:** 622400014

GitHub repository: [Paste repository URL]

1. Objective and data source

In this lab I looked at the January 2021 NYC Yellow Taxi trips file, profiled it to find data quality problems, and wrote a `clean(df)` function for the rules in the assignment. As the assignment says, I stopped before loading anything into PostgreSQL.

Source: https://github.com/DataTalksClub/nyc-tlc-data/releases/download/yellow/yellow_tripdata_2021-01.csv.gz

Each row is one taxi trip. The file has 18 columns (vendor, pickup/dropoff times, passenger count, distance, location IDs, payment type and the fare components). I ran everything in a Jupyter notebook with pandas 2.3.3.

2. Pipeline flow

Stage	What happens here
Source	The compressed January 2021 Yellow Taxi CSV from the DataTalksClub copy of the NYC TLC data.
Extract	The notebook downloads the file if it is missing, then reads it with pandas in chunks of 100,000 rows so the whole file is never in memory at once.
Transform	Profiling collects dtypes, null counts and percentages, min/max values and counts for each quality rule. `clean(df)` then applies the three required cleaning rules.
Load	In class, `ingest_data.py` and Docker Compose load the data into PostgreSQL. In this homework I did not load the cleaned data.
Serve	In class the loaded table can be queried with pgcli or pgAdmin. Nothing is served in this homework.

Why chunks, Int64, and replace on the first chunk only?

Reading in chunks keeps memory low, because only 100,000 rows are held at a time. Nullable integers like pandas `Int64` can hold whole numbers and missing values together. A normal `int64` column cannot, so pandas silently turns it into float when NaN shows up. In the ingestion script the table is replaced on the first chunk so each run starts from a fresh table, and the remaining chunks are appended. If it replaced on every chunk, each new chunk would wipe the previous ones and only the last chunk would be left.

3. In-class pipeline evidence

دوكر مش راضي يشتغل عندي وحاولت كتير بس مش عارف ايه المشكلة ف عملته على vs code

4. Data profiling findings

First I read the first 100 rows to check the structure (`df.head()` and `df.dtypes`). Then I scanned the full file in 14 chunks (13 of 100,000 rows and a last one of 69,765). The total was **1,369,765 rows**, which matches the class reference count.

[SCREENSHOT: `df.head()` and `df.dtypes` output]

[SCREENSHOT: chunk scan output with total rows]

Column profile

Column	Dtype	Nulls	Null %	Min	Max
VendorID	int64	98,352	7.18	1	2
tpep_pickup_datetime	object	0	0	2008-12-31 23:05:14	2021-02-22 16:52:16
tpep_dropoff_datetime	object	0	0	2008-12-31 23:07:22	2021-02-22 16:56:15
passenger_count	int64	98,352	7.18	0	8
trip_distance	float64	0	0	0.0	263,163.28
RatecodeID	int64	98,352	7.18	1	99
store_and_fwd_flag	object	98,352	7.18	N	Y
PULocationID	int64	0	0	1	265
DOLocationID	int64	0	0	1	265
payment_type	int64	98,352	7.18	1	4
fare_amount	float64	0	0	-490.0	6,960.5
extra	float64	0	0	-5.5	8.25
mta_tax	float64	0	0	-0.5	0.5
tip_amount	float64	0	0	-100.0	1,140.44
tolls_amount	float64	0	0	-31.12	811.75

Column	Dtype	Nulls	Null %	Min	Max
improvement_surcharge	float64	0	0	-0.3	0.3
total_amount	float64	0	0	-492.8	7,661.28
congestion_surcharge	float64	0	0	-2.5	3.0

[SCREENSHOT: column profile table]

Data-quality counts

Issue	Count	Share of 1,369,765 rows
Pickup outside January 2021 or invalid	24	about 0.002%
fare_amount <= 0	7,411	about 0.54%
trip_distance <= 0	19,952	about 1.46%
passenger_count = 0	26,726	about 1.95%
passenger_count is null	98,352	7.18%
Exact duplicate rows (hash-based)	0	0%

```
=== df.head() ===
   VendorID tpep_pickup_datetime tpep_dropoff_datetime  passenger_count  \
0         1  2021-01-01 00:30:10   2021-01-01 00:36:12                1
1         1  2021-01-01 00:51:20   2021-01-01 00:52:19                1
2         1  2021-01-01 00:43:30   2021-01-01 01:11:06                1
3         1  2021-01-01 00:15:48   2021-01-01 00:31:01                0
4         2  2021-01-01 00:31:49   2021-01-01 00:48:21                1

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID  DOLocationID  \
0           2.10           1                  N           142            43
1           0.20           1                  N           238           151
2          14.70           1                  N           132           165
3          10.60           1                  N           138           132
4           4.94           1                  N            68            33

   payment_type  fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
0             2          8.0    3.0      0.5        0.00           0.0
1             2          3.0    0.5      0.5        0.00           0.0
2             1         42.0    0.5      0.5        8.65           0.0
3             1         29.0    0.5      0.5        6.05           0.0
4             1         16.5    0.5      0.5        4.06           0.0

   improvement_surcharge  total_amount  congestion_surcharge
0                    0.3         11.80                   2.5
1                    0.3          4.30                   0.0
...
congestion_surcharge     float64
dtype: object

Preview shape: (100, 18)
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```
Chunk 1: 100,000 rows | cumulative rows: 100,000
Chunk 2: 100,000 rows | cumulative rows: 200,000
Chunk 3: 100,000 rows | cumulative rows: 300,000
Chunk 4: 100,000 rows | cumulative rows: 400,000
Chunk 5: 100,000 rows | cumulative rows: 500,000
Chunk 6: 100,000 rows | cumulative rows: 600,000
Chunk 7: 100,000 rows | cumulative rows: 700,000
Chunk 8: 100,000 rows | cumulative rows: 800,000
Chunk 9: 100,000 rows | cumulative rows: 900,000
Chunk 10: 100,000 rows | cumulative rows: 1,000,000
Chunk 11: 100,000 rows | cumulative rows: 1,100,000
Chunk 12: 100,000 rows | cumulative rows: 1,200,000
C:\Users\DELL\AppData\Local\Temp\ipykernel_1676\1283755613.py:21: DtypeWarning: Columns (6) have mixed types. Specify dtype option on import or set low_me
  for chunk_number, chunk in enumerate(reader, start=1):
Chunk 13: 100,000 rows | cumulative rows: 1,300,000
Chunk 14: 69,765 rows | cumulative rows: 1,369,765

Total rows scanned: 1,369,765
Reference count from class slides: 1,369,765
```

column	dtype	null_count	null_percent	min	max
VendorID	int64	98352	7.1802	1	2
tpep_pickup_datetime	object	0	0.0000	2008-12-31 23:05:14	2021-02-22 16:52:16
tpep_dropoff_datetime	object	0	0.0000	2008-12-31 23:07:22	2021-02-22 16:56:15
passenger_count	int64	98352	7.1802	0	8
trip_distance	float64	0	0.0000	0.0	263163.28
RatecodeID	int64	98352	7.1802	1	99
store_and_fwd_flag	object	98352	7.1802	N	Y
PULocationID	int64	0	0.0000	1	265
DOLocationID	int64	0	0.0000	1	265
payment_type	int64	98352	7.1802	1	4
fare_amount	float64	0	0.0000	-490.0	6960.5
extra	float64	0	0.0000	-5.5	8.25
mta_tax	float64	0	0.0000	-0.5	0.5
tip_amount	float64	0	0.0000	-100.0	1140.44
tolls_amount	float64	0	0.0000	-31.12	811.75
improvement_surcharge	float64	0	0.0000	-0.3	0.3
total_amount	float64	0	0.0000	-492.8	7661.28
congestion_surcharge	float64	0	0.0000	-2.5	3.0

issue	count
Pickup outside January 2021 or invalid	24
fare_amount <= 0	7411
trip_distance <= 0	19952
passenger_count = 0	26726
passenger_count is null	98352
Exact duplicate rows (hash-based)	0

Saved profiling tables to: C:\Users\DELL\Desktop\L4-Semster 1\Data Engineering\Lab3_NYC_Taxi_Project.ipynb\Lab3_NYC_Taxi\outputs

What stood out

- Five columns (VendorID, passenger_count, RatecodeID, store_and_fwd_flag, payment_type) all have exactly 98,352 nulls. Since the number is identical, it looks like the same rows are missing all five fields, but I did not check that row by row.
- The pickup dates go back to 2008-12-31 and forward to 2021-02-22, even though this is supposed to be a January 2021 file. Only 24 rows are affected.
- The maximum `trip_distance` is 263,163.28 miles, which is impossible for a taxi trip in NYC, so it is almost certainly a recording error.
- `fare_amount` goes as low as -490 and as high as 6,960.5. Negative amounts also appear in tip, tolls, extra, mta_tax and total_amount, so they may be refunds or adjustments, but the data does not say.
- `passenger_count = 0` (26,726 rows) and `trip_distance <= 0` (19,952 rows) are much more common than the other problems.
- The hash check found no exact duplicates. Because it uses 64-bit row hashes, a collision is possible in theory, but very unlikely.
- The profile table shows `passenger_count` as int64 even though it has 98,352 nulls. The dtype in my script is taken from the first chunk, and that chunk had no nulls. This is the same int64 vs. Int64 issue mentioned in section 2.

5. Cleaning decisions

Problem / rule	Decision	Reason / owner
Null passenger_count	Drop	Required by the assignment. The data-quality or analytics owner should confirm whether these rows are worth keeping for other analysis.
Pickup timestamp invalid or outside Jan 2021	Drop	The file is a monthly January 2021 extract, so other dates break the expected period.
fare_amount <= 0	Drop	Required by the assignment. Zero or negative fares need a business decision (refunds, voids, test trips), so the owner should review them.
trip_distance <= 0	Profile and flag	Counted in Task 1, but Task 2 does not ask to remove it, so I did not add an extra rule.
passenger_count = 0	Profile and flag	Same reason. It stays unless a business rule says otherwise.
Exact duplicates	Count and report	There were none, and Task 2 does not ask for duplicate removal.

`clean(df)` works on a copy, so the original DataFrame is not modified. It applies the three rules one after another and returns the cleaned DataFrame together with a dictionary of rows removed per rule. Doing it in sequence means a row that breaks two rules is only counted once, under the first rule that catches it.

```
Rows before: 100,000
Rows after: 99,263
Rows removed total: 737
Rows removed per rule:
{'null_passenger_count': 0, 'pickup_outside_january_2021_or_invalid': 16, 'fare_amount_less_than_or_equal_to_zero': 721}
Original input unchanged: True
```

```
Chunk 1: 100,000 rows | cumulative rows: 100,000
Chunk 2: 100,000 rows | cumulative rows: 200,000
Chunk 3: 100,000 rows | cumulative rows: 300,000
Chunk 4: 100,000 rows | cumulative rows: 400,000
Chunk 5: 100,000 rows | cumulative rows: 500,000
Chunk 6: 100,000 rows | cumulative rows: 600,000
Chunk 7: 100,000 rows | cumulative rows: 700,000
Chunk 8: 100,000 rows | cumulative rows: 800,000
Chunk 9: 100,000 rows | cumulative rows: 900,000
Chunk 10: 100,000 rows | cumulative rows: 1,000,000
Chunk 11: 100,000 rows | cumulative rows: 1,100,000
Chunk 12: 100,000 rows | cumulative rows: 1,200,000
C:\Users\DELL\AppData\Local\Temp\ipykernel_1676\1283755613.py:21: DtypeWarning: Columns (6) have mixed types. Specify dtype option on import or set low_mem
  for chunk_number, chunk in enumerate(reader, start=1):
Chunk 13: 100,000 rows | cumulative rows: 1,300,000
Chunk 14: 69,765 rows | cumulative rows: 1,369,765

Total rows scanned: 1,369,765
Reference count from class slides: 1,369,765
```

Demonstration on the first chunk

To keep memory low I ran `clean()` on the first 100,000-row chunk only.

Measure	Value
Rows before	100,000
Rows after	99,263
Total removed	737
Removed: null passenger_count	0
Removed: pickup outside Jan 2021 or invalid	16
Removed: fare_amount <= 0	721

The check `first_chunk.equals(original_first_chunk)` returned `True`, so the input was not changed.

The null rule removed 0 rows here because the nulls in this file only start appearing in later chunks. That is why the full-file count (98,352) is so much larger than what this demo shows.

[SCREENSHOT: rows before/after and removed_rows dictionary]

6. Wrap-up questions

Q1. Why does the container use `pgdatabase` rather than `localhost`?

Inside a container, `localhost` points to that same container. PostgreSQL runs in a different container, so the ingestion container has to reach it by its Docker Compose service name, `pgdatabase`, which Docker resolves on the shared network.

Q2. Why use chunks, and what is wrong with `replace` for many months?

Chunks keep memory usage small because the whole file is never loaded at once. Using `replace` on every chunk (or every month) is a problem because each replace drops the table and recreates it, so everything loaded before is lost. `replace` should only be used once to create the table at the start, and later chunks or months should be appended.

Q3. How did you verify the results?

- The total rows from my chunked scan (1,369,765) matched the class reference count.
- I read the printed profile and issue-count tables and checked that they made sense, for example that the min/max values show the odd dates and distances I listed above.
- For `clean()` I checked on the first chunk that rows before minus rows after equals the sum of the per-rule counts: 100,000 − 99,263 = 737 = 0 + 16 + 721. I also confirmed the original DataFrame was unchanged.
- I only ran the cleaning demo on the first chunk, not on the whole file.

7. What broke and how it was fixed

Nothing crashed in the notebook, but I did get one warning during the full scan. Pandas printed a `DtypeWarning` saying column 6 (`store_and_fwd_flag`) has mixed types. This happens because pandas guesses the dtype separately for pieces of the file when reading, and that column holds both strings and missing values. It did not change the results, since the column is only used for null counts and min/max (N and Y). To remove the warning I could pass `dtype={"store_and_fwd_flag": "string"}` or set `low_memory=False` in `read_csv`.

The second thing I ran into was the `passenger_count` dtype issue described in section 4, where the profile showed int64 even though the column has nulls.